

La similitud encuentra el hecho, no la versión

Un sello de orden co-entrenado en la clave resuelve el conflicto de versiones en una memoria persistente dentro del modelo

Maximiliano Speranza
Investigador independiente, Buenos Aires, Argentina
[ORCID 0009-0005-0413-8554](https://orcid.org/0009-0005-0413-8554)
maximiliano.speranza@gmail.com

Septiembre de 2026

Resumen

Un archivo persistente co-entrenado adentro de un modelo de lenguaje aprende a recuperar el hecho correcto y después no sabe cuál versión de ese hecho rige. Esa falla está medida con precisión. Con una sola versión archivada el modelo responde 0,9974, y con dos versiones de la misma clave compitiendo cae a 0,4576, que es el azar entre el valor viejo y el nuevo. La recuperación queda intacta y lo que falta es el orden.

Mostramos que la información faltante entra en un **sello de orden** chico y co-entrenado, agregado a la clave del archivo, y que lo que hace el trabajo es el ordenamiento y no la capacidad extra. Sobre cinco semillas, las claves revisadas pasan de $0,4570 \pm 0,0289$ sin sello a $0,9956 \pm 0,0029$ con sello. El control de falsación, un sello con los mismos parámetros cuyo valor no guarda relación con el turno real de escritura, da $0,4768 \pm 0,0389$, adentro de una desviación estándar del piso. Las quince corridas se ordenan perfecto por condición, sin un solo solape entre celdas.

Otros tres resultados precisan la afirmación. Primero, con tres versiones por clave, cada una escrita en su propia secuencia para que no sobreviva ningún rastro de recencia en el contenido, preguntar por el valor superado pasa de 0,2917, que es el azar entre tres, a 0,8316. Segundo, cuando los turnos de escritura se sortean al azar y ninguna tabla fija de posición puede resolver la tarea, el sello igual da 0,9644 en la versión vigente, lo que descarta ese atajo. Tercero, y es lo más útil en la práctica, un sello *corrupto* es peor que no tener sello, y degrada incluso la identificación del ítem, que baja de 0,9974 a 0,8802. Una marca de tiempo equivocada no es información faltante sino información dañina.

También separamos dos capacidades que suelen confundirse. Preferir el valor más reciente y usar el orden se aprenden en momentos distintos, y la vigente satura a los 3000 pasos mientras el valor anterior está todavía en cero exacto. Un banco de pruebas que sólo pregunta por el valor vigente no puede ver esa distinción, porque el atajo de la recencia ya le da la respuesta correcta.

1. El problema, medido

Las memorias de atención lineal y de regla delta tienen un estado recurrente de tamaño fijo y sobrescriben al escribir, así que lo viejo se pierde por construcción (Schlag et al., 2021; Yang et al., 2025). La reparación natural es un archivo externo, y una línea de trabajo creciente co-entrena ese archivo adentro del modelo en vez de enchufarle un recuperador congelado (Feldman et al., 2026; Jeong, 2026; Zhao et al., 2025).

Esa reparación funciona para recuperar y deja un agujero específico. En nuestra etapa previa, un archivo co-entrenado que guarda vectores producidos por el propio modelo llega a 0,9974 cuando una clave tiene una sola versión archivada, y se derrumba a 0,4576 cuando esa misma clave tiene dos versiones compitiendo. El valor 0,4576 no es ruido, es el azar entre la vieja y la nueva. El modelo encuentra el hecho correcto y no sabe cuál versión rige.

Es el mismo modo de falla que ya habíamos medido en un índice no paramétrico, donde la geometría agrupaba las versiones de un recuerdo perfecto y recuperaba la *más vieja*. Dos mecanismos sin nada en común fallan igual, lo que sugiere que el diagnóstico es una propiedad de la recuperación por similitud y no de una implementación particular. La similitud identifica el ítem. El orden es información que la similitud no contiene.

2. Trabajo relacionado

Híbridos intra-secuencia. Cui (2026) combina un estado de atención lineal con una memoria exacta para lo que ese estado olvida, y Lufkin et al. (2026) junta un estado recurrente con atención que lo complementa sólo donde el estado predice mal. Los dos son híbridos de un estado compresivo con un almacén exacto, y en los dos ese almacén vive adentro de un solo paso hacia adelante, así que la pregunta de cuál de dos versiones escritas en sesiones *distintas* rige no puede aparecer.

Memoria persistente co-entrenada. Feldman et al. (2026) es lo más cercano en mecanismo, porque emite consultas continuas desde los estados ocultos del modelo para recuperar de una base de conocimiento, con 135M a 360M de parámetros. Jeong (2026) acumula un banco de memoria entre turnos sobre un GPT-2 congelado, de modo que hechos dichos en una sesión se recuerdan después. Ninguno versiona un hecho, ninguno pone orden en la clave, y ninguno pregunta jamás por un valor superado. Co-LMLM trata la base como una foto estática, y Jeong (2026) aplica un decaimiento temporal uniforme, que atenúa lo viejo en vez de ordenarlo.

Interferencia proactiva. Wang y Sun (2025) es lo más cercano en la *tarea*, porque actualiza claves varias veces adentro del contexto y pide el valor vigente con las versiones viejas presentes como interferencia. Documenta el mismo atajo de recencia que encontramos nosotros. Es un banco de pruebas y un diagnóstico, sus autores declaran que no ofrecen mitigación arquitectónica, evalúa sólo modelos grandes preentrenados, y nunca pide el valor anterior. Ese último punto nos importa, porque el atajo de la recencia responde solo la pregunta por el valor vigente.

Versionado en capa de aplicación. Existen sistemas que versionan documentos con metadatos y punteros explícitos, en cañerías de recuperación montadas sobre un modelo congelado. El sello que estudiamos no es eso. Vive adentro de la clave de recuperación del propio modelo y se entrena junto al lector, en vez de estar en un campo de metadatos que consume un recuperador aparte.

Marcas de tiempo en la entrada. Rajaby Faghihi y Kordjamshidi (2021) inyecta marcas temporales en la entrada para flujos de eventos, pasado, presente y futuro. Eso no es dos versiones del mismo hecho adentro de una memoria, y la marca no entra en ninguna clave de recuperación.

3. Montaje experimental

El sustrato es un modelo de lenguaje chico con un archivo co-entrenado, 64 dimensiones ocultas, un archivo de 9 entradas y una tarea sintética entre secuencias. Los hechos se escriben en una secuencia, se revisan en otra y se consultan en una tercera, **con el estado recurrente reseteado entre secuencias**. La recuperación entonces no se puede resolver arrastrando activaciones, y el piso es el azar por construcción.

El archivo guarda vectores producidos por el modelo, no tokens. Escribir la entrada i desde el estado h_i da la clave $k_i = h_i W_k$ y el valor $v_i = h_i W_v$, y leer forma una consulta desde el estado que consume y atiende sobre el archivo. El **sello** agrega a la clave un embedding aprendido del turno de escritura,

$$k_i = h_i W_k + \text{ord}[t_i],$$

donde t_i es el turno en que se escribió la entrada i y ord es una tabla entrenada. Son 384 parámetros efectivos. Y algo decisivo, **el archivo se baraja por muestra** y cada sello viaja

pegado a su propia entrada. Sin barajar, el turno de escritura coincidiría siempre con la posición en el tensor y el experimento mediría mirar el final del archivo, que es posición y no orden.

Se comparan tres condiciones en todo el trabajo. **ninguno** no tiene sello. **sello** es lo de arriba. **barajado** es el control de falsación, con una tabla del mismo tamaño y entrenada igual, cuyo valor de sello *no guarda relación* con el turno real de escritura.

4. Resultado 1. El sello levanta el conflicto de versiones, y es el orden lo que lo levanta

Cinco semillas, dos versiones por clave revisada, 4000 pasos.

condición	global	claves revisadas	claves no revisadas
ninguno	$0,7273 \pm 0,0146$	$0,4570 \pm 0,0289$	0,9977
sello	$0,9970 \pm 0,0020$	$0,9956 \pm 0,0029$	0,9984
barajado	$0,7374 \pm 0,0194$	$0,4768 \pm 0,0389$	0,9979

La ganancia sobre no tener sello es de +0,5385 y sobre el control barajado de +0,5187. **Las cinco semillas con sello caen entre 0,9909 y 0,9987, y la mejor semilla de las otras dos condiciones llega a 0,5156. Las quince corridas se ordenan perfecto por condición, sin un solo solape.** Con esa distancia, el intervalo de confianza es un trámite.

Lo que hace válido el resultado es la celda que no ganó. La condición **barajado** tiene exactamente los mismos parámetros que **sello**, entrenados igual, y lo único que cambia es que el sello no guarda relación con el turno real. Sin esa celda, el +0,5385 se explicaría igual de bien diciendo que se le agregaron parámetros al lector. Con ella, la explicación por capacidad queda descartada de forma directa.

La aritmética es simétrica. El conflicto de versiones costaba $-0,5398$ y el sello devuelve +0,5385. Repone lo que la similitud había perdido y nada más, porque las claves no revisadas se quedan en 0,998, donde ya estaban.

5. Resultado 2. Tres versiones, y preguntar por el valor superado

Un primer intento con tres versiones se rompió solo, y cómo se rompió vale registrarlo. Al escribir las dos revisiones adentro de la misma secuencia, el estado en v_3 arrastraba un rastro de v_2 , así que el orden quedaba escrito en el *contenido* sin que ningún metadato lo pusiera. A 4000 pasos era invisible porque ninguna condición resolvía la pregunta difícil, y a 12000 pasos la condición sin sello llegó a 0,97 y delató la fuga.

Con cada versión escrita en su propia secuencia y el estado reseteado entre ellas, v_2 y v_3 son cada una primera y única mención de su secuencia, indistinguibles en recencia. Lo único en el sistema que dice cuál vino antes es el sello. Tres semillas, 12000 pasos.

condición	vigente	anterior	claves de una sola versión
ninguno	0,3090	$0,2917 \pm 0,0207$	0,9436
sello	0,9774	$0,8316 \pm 0,2399$	0,9974
barajado	0,2682	$0,2804 \pm 0,0159$	0,8802

Sin sello el modelo se queda en 1/3, el azar entre tres versiones, y al mismo tiempo identifica bien el ítem cuando no hay conflicto, con 0,9436. La fuga quedó tapada, y la distancia respecto del 0,97 del diseño roto mide exactamente cuánta información de orden se estaba colando por el contenido.

Este número depende del presupuesto y lo informamos así. Con cinco semillas el valor anterior baja a 0,7667 con desviación 0,2340, y sus valores son 0,9766, 0,9635, 0,5547, 0,8594 y 0,4792. El mecanismo no falla, sino que dos de cinco semillas no convergieron a 12000 pasos. Se probó de forma directa, y la semilla 2 extendida a 24000 pasos pasó de 0,5547 a 0,9531. El veredicto honesto es condicional, y dice que el sello permite responder por la versión anterior con un presupuesto que 12000 pasos no siempre alcanza. **En este régimen el tiempo de convergencia es parte del fenómeno y no un detalle de implementación.**

6. Resultado 3. No es una tabla de posiciones

En el diseño anterior los turnos de escritura eran fijos por rol, así que el modelo podía aprender que los embeddings 9, 10 y 11 significan versión vigente. Eso es una correspondencia entre posición y rol, no un orden, y el control barajado no las separa, porque aleatorizar el sello rompe la tabla igual que rompe el orden.

Entonces sorteamos doce turnos al azar de un rango de treinta y dos por muestra, los ordenamos y los asignamos respetando el orden real de escritura. El turno 9 puede ser una primera versión en una muestra y una tercera en la siguiente, de modo que ninguna tabla fija resuelve la tarea. Tres semillas.

condición	vigente	anterior
ninguno	0,3247	$0,2977 \pm 0,0326$
sello	0,9644	$0,3142 \pm 0,5330$
barajado	0,2804	$0,2925 \pm 0,0266$

El resultado en la versión vigente sobrevive a los turnos móviles, lo que **descarta el atajo de la tabla y valida hacia atrás el Resultado 1**, porque si el mecanismo hubiera sido una tabla, mover los turnos habría roto también la vigente.

La columna del valor anterior es **bimodal, no ruidosa**, con valores 0,0052, 0,9297 y 0,0078. Dos semillas caen en un atajo de recencia y responden *por debajo* del azar, porque devuelven la versión vigente cuando se pide la anterior, y una semilla aprende la comparación completa. Informamos la distribución y no la media, porque el promedio de 0,31 no describe a ninguna de las tres corridas. El resumen honesto es que el lector aprende a leer el reloj para saber qué hora es ahora, no para reconstruir la secuencia de lo que pasó.

7. Resultado 4. Un sello mentiroso es peor que ninguno

Leer la fila **barajado** del Resultado 2 como un resultado en sí mismo, y no sólo como control, da el hallazgo con más filo práctico. Un sello corrupto deja la versión vigente en 0,2682, por *debajo* del 0,3090 de no tener sello, y degrada la identificación del ítem de 0,9974 a 0,8802, que es una capacidad que no tiene nada que ver con ordenar.

El lector se apoya en el sello. Una vez que lo hace, una marca de tiempo corrupta no es información faltante sino información dañina, y el daño se contagia a la calidad de la recuperación. Para cualquier sistema que le ponga marca de tiempo a sus entradas de memoria, esto dice que un sello viejo o equivocado es peor que un campo ausente.

8. Dos capacidades que se aprenden en momentos distintos

Preferir el valor más reciente y usar el orden son cosas distintas y separables, y el orden entre las dos es estable entre réplicas aunque el momento no lo sea. En las dos réplicas con tasa de aprendizaje 10^{-3} la versión vigente satura a los 3000 pasos, con 0,9583 en ambas, mientras

el valor anterior está todavía en 0,0000. Después el valor anterior despega, y cuánto más tarde varía. Una réplica llega a 0,2500 a los 4000 pasos, 0,6250 a los 5000 y 0,9583 a los 6000, mientras que la otra a los 4000 está en 0,0208 y no pasa 0,9 hasta los 8000. Informamos las dos porque el promedio no describiría a ninguna. Lo que replica es el **orden**, ya que en toda corrida la versión vigente se resuelve miles de pasos antes que la anterior, y en el paso donde la vigente ya está en 0,9583 la anterior está en cero exacto.

Esto tiene una consecuencia directa para la evaluación. **Un banco de pruebas que sólo pregunta por el valor vigente no puede ver la distinción**, porque el atajo de la recencia da la respuesta correcta. Es precisamente lo que Wang y Sun (2025) no hace, y por eso consideramos que la pregunta por el valor anterior es la sonda más informativa.

9. Límites

Es suficiencia, no inferencia. En esta tarea la versión vigente es siempre la del turno más alto, así que la política aprendible es quedarse con el sello más grande. El resultado dice que la información que a la similitud le falta *entra en un sello* y que el lector la sabe usar. No dice que el modelo razone sobre el tiempo.

El sello se le da, no se infiere. Mostramos que el lector puede usar el metadato, no que pueda fabricarlo. No es un oráculo irreal, porque el turno de escritura es gratis para cualquier índice, a diferencia de saber cuál versión es la correcta, pero acota la afirmación.

Escala. Un modelo de 64 dimensiones, un archivo de 9 entradas y una tarea sintética. La dificultad medida es de alineación y de conflicto de versiones, no de discriminación entre muchos candidatos.

Un archivo fresco. En estos experimentos el archivo se recomputa en cada paso. Un trabajo posterior donde las entradas envejecen muestra que el daño es gradual y no un acantilado, y las claves revisadas bajan de 0,9987 a 0,9115 a medida que el coseno entre el marco de escritura y el de lectura cae a 0,78.

10. Conclusión

La similitud identifica el ítem, y el orden es información que la similitud no contiene y que hay que llevar aparte. En una memoria persistente co-entrenada adentro de un modelo de lenguaje chico, llevarla como un sello en la clave lleva el conflicto de versiones del azar a casi el techo, y un control con los mismos parámetros y un sello sin significado muestra que lo que trabaja es el ordenamiento y no la capacidad. El mismo mecanismo vuelve responsable la pregunta por el valor superado, que ningún banco de pruebas del área hace hoy, y viene con una advertencia que se generaliza más allá de nuestro montaje. Una vez que un lector aprende a apoyarse en un sello, un sello equivocado cuesta más que no tener ninguno.

Reproducibilidad y disponibilidad de datos

Cada experimento acá informado fue pre-registrado con sus predicciones y umbrales hasheados antes de la corrida correspondiente. Las celdas que no cumplieron sus criterios se informan como incumplimientos, incluido el valor anterior dependiente del presupuesto del Resultado 2 y la columna bimodal del Resultado 3. Todos los pre-registros con sus hashes SHA, los scripts de análisis, los resultados crudos por semilla y los informes generados por máquina están archivados en el repositorio del proyecto, <https://github.com/SperanzaMax/delta-archivo>, cuyo historial de commits lleva las marcas de tiempo que establecen el orden entre pre-registro y resultado.

Declaraciones

Financiamiento. Ninguno. **Intereses en competencia.** Ninguno. **Aprobación ética.** No aplica, no hubo sujetos humanos ni animales. **Disponibilidad de datos.** Ver arriba. **Contribución de autoría.** Autor único.

Referencias

- Cui, W. (2026). A Hippocampus for Linear Attention. An Exact Memory for What the Recurrent State Forgets. arXiv:2607.02303.
- Lufkin, L., Figliolia, T., Millidge, B., y Krishnamurthy, K. (2026). Hybrid Associative Memories. arXiv:2603.22325.
- Feldman, Y., Zhao, L., Godey, N., Go, D., Hua, Y., Weinberger, K. Q., Sun, J. J., y Artzi, Y. (2026). Co-LMLM. Continuous-Query Limited Memory Language Models. arXiv:2607.07707.
- Jeong, H. (2026). Trained Persistent Memory for Frozen Decoder-Only LLMs. arXiv:2603.22329.
- Zhao, L., Zalouk, S., Belardi, C. K., Lovelace, J., Zhou, J. P., Noonan, R. T., Go, D., Weinberger, K. Q., Artzi, Y., y Sun, J. J. (2025). Pre-training Limited Memory Language Models with Internal and External Knowledge. arXiv:2505.15962.
- Wang, C., y Sun, J. V. (2025). Unable to Forget. Proactive Interference Reveals Working Memory Limits in LLMs Beyond Context Length. arXiv:2506.08184.
- Schlag, I., Irie, K., y Schmidhuber, J. (2021). Linear Transformers Are Secretly Fast Weight Programmers. ICML 2021.
- Yang, S., Kautz, J., y Hatamizadeh, A. (2025). Gated Delta Networks. Improving Mamba2 with Delta Rule. ICLR 2025. arXiv:2412.06464.
- Rajaby Faghihi, H., y Kordjamshidi, P. (2021). Time-Stamped Language Model. Teaching Language Models to Understand the Flow of Events. NAACL 2021. arXiv:2104.07635.